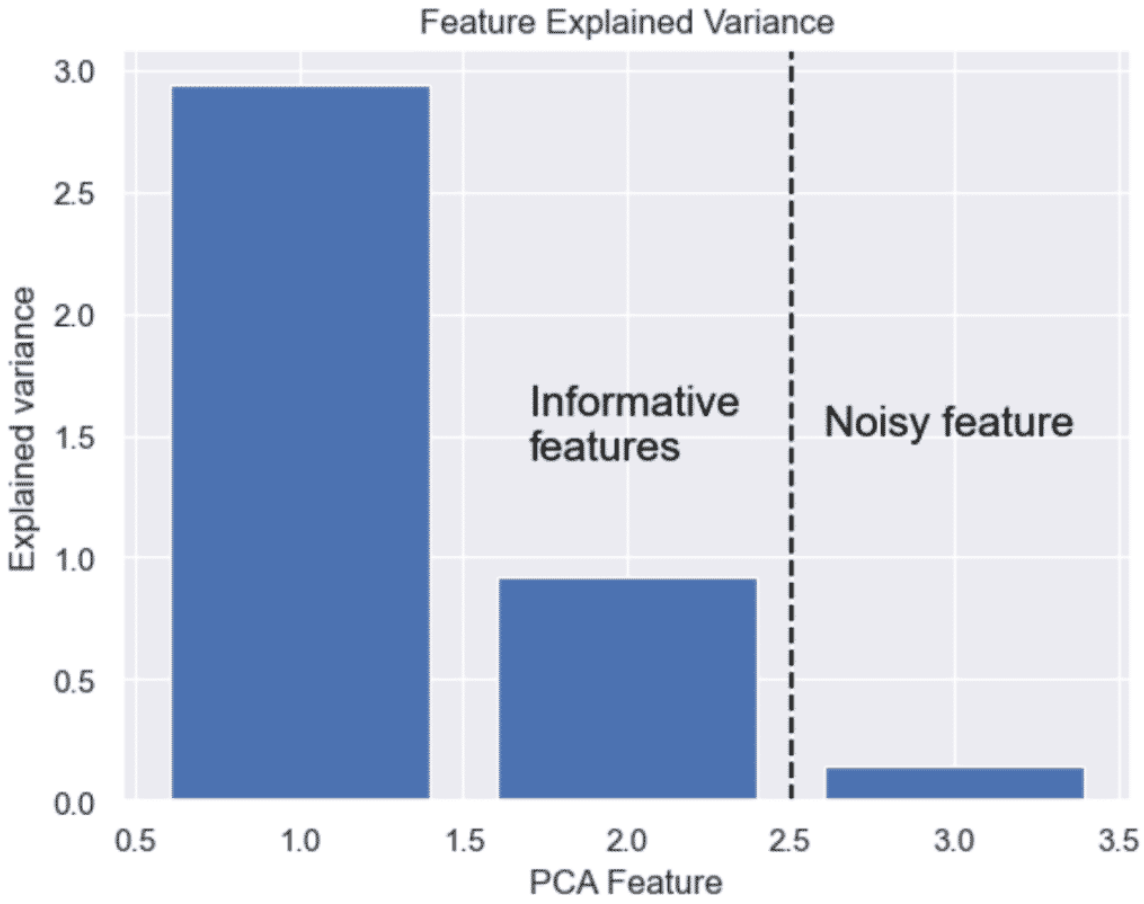
WEEK 03- PCA MACHINE LEARNING ALGORITHM

What is Principal Component Analysis (PCA)?

PCA, or Principal component analysis, is the main linear algorithm for dimension reduction often used in unsupervised learning. In simple words, PCA tries to reduce the number of dimension whilst retaining as much variation in the data as possible.

This algorithm identifies and discards features that are less useful to make a valid approximation on a dataset.



Introduction to PCA in Python

Principal Component Analysis (PCA) is a technique used in Python and machine learning to reduce the dimensionality of high-dimensional data while preserving the most important information.

Simply put, PCA makes complex data simpler by taking a lot of information and finding the most important parts. This helps to fight the curse of dimensionality.

Install Scikit-Learn to use PCA in Python

For this tutorial, you will also need to install Python and install Scikit-learn library from your command prompt or Terminal.

$ pip install scikit-learn

Simplest Example of PCA in Python

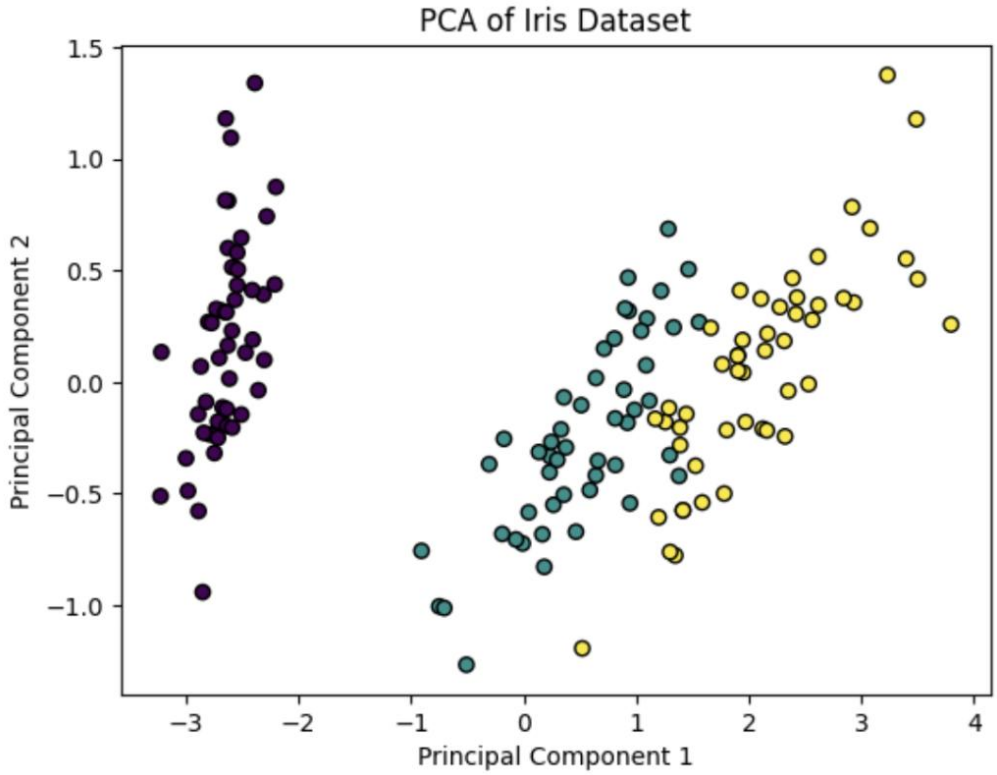
Here is a simple example of how to use Python PCA algorithm in Scikit-learn to reduce the features of the Iris dataset and plot a 2D graph.

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Apply PCA with two components (for 2D visualization)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Plot the results
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k')
plt.title('PCA of Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```



Getting Started with Principal Component Analysis in Python

In this Python tutorial, we will perform principal component analysis on the Iris dataset using Scikit-learn. We will now install Scikit-learn and load the built-in Iris dataset.

1. **Explore** the Iris Dataset
2. **Load the Dataset** with Sciki-learn
3. **Perform Data Preprocessing** in Python
4. **Perform Dimension Reduction** using PCA in Python
5. **Give Names for Your Plot** by Mapping targets to Principal Components
6. **Plot** the 2D PCA Graph

1. Explore the Python Dataset (Iris)

The Iris dataset is useful to visualize how Principal Component Analysis works.

The iris dataset contains 4 features (predictor variables) describing 3 species of flowers (targets).

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_names
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9			0.2	0	
2	4.7			0.2	0	
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

source: iris dataset in pandas

2. Load the Iris Dataset with Scikit-Learn

To load the Iris dataset from Scikit-learn, let's load features and targets as arrays stored in their respective X and y variables.

```
1from sklearn import datasets

2

3# load features and targets separately

4iris = datasets.load_iris()

5X = iris.data

6y = iris.target
```

```
X
✓ 0.9s
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
```

3. Perform Data Preprocessing in Python

Before running PCA, let's perform very basic data preprocessing and scale the data using StandardScaler.

```
1# data scaling
2from sklearn.preprocessing import StandardScaler
3x_scaled = StandardScaler().fit_transform(X)
```

StandardScaler will standardize the features by removing the mean and scaling to unit variance so that each feature has $\mu = 0$ and $\sigma = 1$.

Converting this:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Unscaled features Into this:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444

Scaled features

4. Perform Dimension Reduction using PCA in Python

To perform dimension reduction in Python, import PCA from sklearn.decomposition and use the fit_transform() method on the PCA() object. The n_components argument tells the number of dimensions to keep.

We have seen that the Iris dataset contains 4 features, making it a 4-dimensional dataset. Not all features are necessarily useful for the prediction. Therefore, we can remove those noisy features and make a faster model.

The n_components argument will define the number of components that we want to reduce the features to.

```
1# Dimention Reduction
2pca = PCA(n_components=2)
3pca_features = pca.fit_transform(x_scaled)
```

```
4

5# Show PCA characteristics

6print('Shape before PCA: ', x_scaled.shape)

7print('Shape after PCA: ', pca_features.shape)
```

```
Shape before PCA:  (150, 4)

Shape after PCA:  (150, 2)
```

For better understanding, PCA converted this data with 4 features.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444

Into this data with 2 principal components.

```
1# Create PCA DataFrame

2pca_df = pd.DataFrame(

3    data=pca_features,

4    columns=[

5        'Principal Component 1',

6        'Principal Component 2'

7        ]
```

	Principal Component 1	Principal Component 2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835
...	...	...
145	1.870503	0.386966
146	1.564580	-0.896687
147	1.521170	0.269069
148	1.372788	1.011254
149	0.960656	-0.024332

150 rows × 2 columns

5. Give Names for Your Plot by Mapping targets to Principal Components

We will prepare the PCA Dataframe for visualization by mapping the target names to the PCA features.

```
# Map target names to targets
target_names = {
    0:'setosa',
    1:'versicolor',
    2:'virginica'
}

pca_df['target'] = y
pca_df['target'] = pca_df['target'].map(target_names)
pca_df.sample(10)
```

	Principal Component 1	Principal Component 2	target
138	0.924825	0.017223	virginica
94	0.288586	-0.855730	versicolor
98	-0.447667	-1.543792	versicolor
103	1.440151	-0.046988	virginica
32	-2.614948	1.793576	setosa
115	1.590077	0.676245	virginica
112	1.883901	0.419250	virginica
36	-2.045146	0.661558	setosa
0	-2.264703	0.480027	setosa
8	-2.334640	-1.115328	setosa

6. Plot the 2D PCA Graph

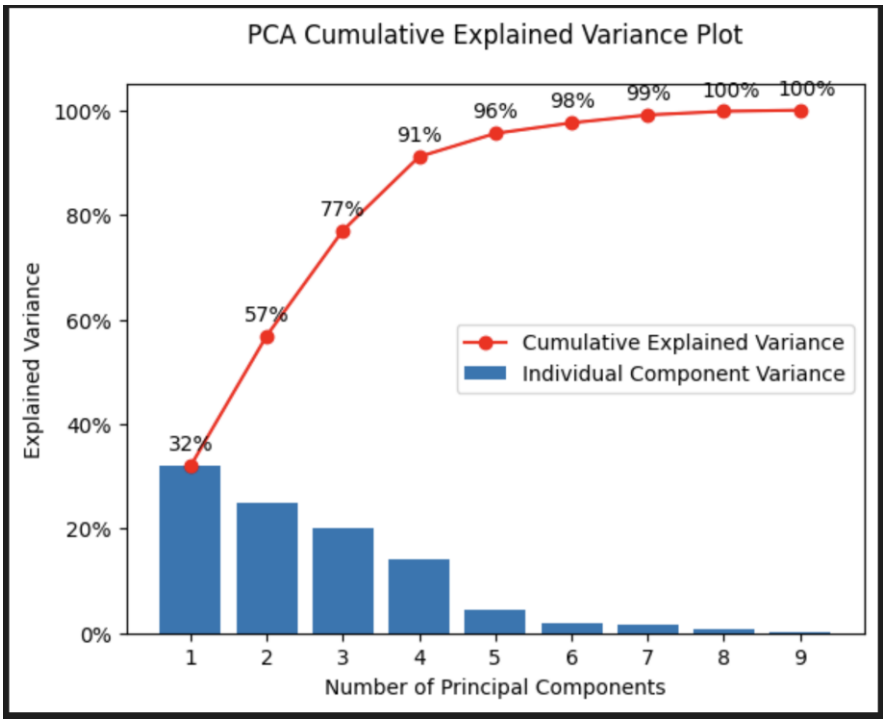
To plot the 2D PCA graph, use the Principal Component DataFrame to Seaborn's lmplot() function.

```
1  import matplotlib.pyplot as plt
2  import seaborn as sns
3  sns.set()
4  # Plot 2D PCA Graph
5  sns.lmplot(
6      x='Principal Component 1',
7      y='Principal Component 2',
8      data=pca_df,
9      hue='target',
10     fit_reg=False,
11     legend=True
12     )
13
14plt.title('2D PCA Graph of Iris Dataset')
15plt.show()
```

How Many Dimensions Should You Reduce Your Data with PCA?

To decide the number of dimensions that you should reduce your data to, you should make a cumulative explained variance plot (scree plot) to show each of your individual component variance on top of your cumulative variance.

We could decide to keep 90%+ of the variance and thus keep 3 Principal components.



Full PCA Code

```
import pandas as pd
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()

# load features and targets separately
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Data Scaling
x_scaled = StandardScaler().fit_transform(X)

# Dimention Reduction
pca = PCA(n_components=2)
pca_features = pca.fit_transform(x_scaled)

# Show PCA characteristics
print('Shape before PCA: ', x_scaled.shape)
print('Shape after PCA: ', pca_features.shape)
print('PCA Explained variance:', pca.explained_variance_)

# Create PCA DataFrame
pca_df = pd.DataFrame(
    data=pca_features,
    columns=[
        'Principal Component 1',
        'Principal Component 2'
        ])


# Map target names to targets
target_names = {
    0:'setosa',
    1:'versicolor',
    2:'virginica'
}

pca_df['target'] = y
pca_df['target'] = pca_df['target'].map(target_names)
pca_df.sample(10)

# Plot 2D PCA Graph
sns.lmplot(
    x='Principal Component 1',
    y='Principal Component 2',
    data=pca_df,
    hue='target',
    fit_reg=False,
    legend=True
    )
```

```
55
56plt.title('2D PCA Graph of Iris Dataset')
57plt.show()
58
59
60# Bar plot of explained_variance
61plt.bar(
62    range(1,len(pca.explained_variance_)+1),
63    pca.explained_variance_
64    )
65
66plt.xlabel('PCA Feature')
67plt.ylabel('Explained variance')
68plt.title('Feature Explained Variance')
69plt.show()
```

Exercises

Q-Use IRIS data set plot the 6 Best PCA Plots & Visualizations with Python such as

1. Feature Explained Variance Bar Plot
2. PCA Scree plot
3. 2D PCA Scatter plot
4. 3D PCA Scatter plot
5. 2D PCA Biplot
6. 3D PCA Biplot